ECG Diffusion Transformer

Generation Research Report

Diffusion vs Flow Matching | ECG-Specific Losses | HR Diversity

Research-Level Technical Analysis
2025

1. Overview

This report provides a research-level technical analysis of Diffusion Transformer (DiT)-based ECG generation, addressing three core challenges: choosing the right generative framework, designing physiologically-aware loss functions, and recovering heart-rate (HR) diversity in generated signals.

System Configuration

Parameter	Value
Signal length	2500 samples @ 500 Hz (5 seconds)
Model backbone	Diffusion Transformer (DiT)
Diffusion process	DDPM with cosine noise schedule
Sampler	DDIM (deterministic)
Current losses	MSE + FFT spectral + Pearson correlation + cosine identity
HR issue	Std HR = 21 bpm (generated) vs 47 bpm (real) - collapsed diversity

The primary problem is mode collapse of HR variability: the model converges to a narrow HR band around the dataset mean instead of spanning the full physiological range. This report identifies root causes and proposes a multi-pronged remediation.

2. Diffusion vs Flow Matching for 1D ECG Generation

Generative frameworks for 1D biosignals can be broadly categorized into diffusion-based (DDPM/DDIM) and flow-based (Flow Matching, Rectified Flow, Conditional Flow Matching) approaches. The choice affects training stability, inference speed, and, critically for biosignals, whether the model preserves sharp spectral features and beat morphology.

2.1 Framework Descriptions

DDPM — Denoising Diffusion Probabilistic Models (Ho et al., 2020)

DDPM defines a fixed Markov chain that gradually adds Gaussian noise over T=1000 steps, then trains a neural network to reverse this process. The forward process is closed-form; the reverse requires iterating T steps at inference. The cosine noise schedule (Nichol & Dhariwal, 2021) provides more stable gradients for 1D signals than the original linear schedule.

- Strengths: Well-understood, stable training, strong theoretical grounding.
- Weaknesses: 1000-step inference; deterministic DDIM collapse (see HR issue).

DDIM — Denoising Diffusion Implicit Models (Song et al., 2020)

DDIM is a non-Markovian sampler on top of a DDPM-trained model. It enables 20-50x inference acceleration by skipping steps. However, because DDIM is deterministic given a fixed noise seed, it systematically reduces sample diversity. This is a primary cause of the HR collapse in your current model.

- Strengths: Fast inference, no retraining required, compatible with any DDPM model.
- Weaknesses: Determinism suppresses stochasticity needed for diversity.

Flow Matching (Lipman et al., 2022)

Flow Matching defines a continuous normalizing flow (CNF) by constructing a vector field that transports samples from a simple noise distribution to the data distribution in a single straight path. Training minimizes the mean squared error between the predicted and target velocity field at randomly sampled time steps. The target velocity is simply v(x,t) = x_data - x_noise (for optimal transport paths), making training extremely stable.

- Strengths: Straight transport paths, fast convergence, O(1) NFE possible.
- Weaknesses: Requires marginal path construction; less literature for ECG.

Rectified Flow (Liu et al., 2022)

Rectified Flow learns straight-line ODE trajectories between noise and data by training the velocity field on paired (noise, data) samples. It can be further distilled (ReFlow) to achieve 1-2 step generation. The straight paths are particularly efficient for 1D signals where interpolation artifacts are minimal.

- Strengths: Extremely fast inference (1-4 steps), stable training, distillable.
- Weaknesses: Requires careful pairing in the data loader.

Conditional Flow Matching (Albergo & Vanden-Eijnden, 2022)

CFM extends Flow Matching with arbitrary conditional probability paths. It is strictly more general than DDPM and subsumes it as a special case. For patient-conditioned ECG generation, CFM allows conditioning on HR, RR intervals, and patient embeddings directly in the velocity field parameterization.

- Strengths: Maximum conditioning flexibility, supports classifier-free guidance, fast sampling.
- Weaknesses: Slightly more complex implementation than DDPM.

2.2 Comparison Table

Method	Convergence	Training Stability	Sample Quality	Inference Speed	1D ECG Fit	DiT Compatible?
DDPM	Slow (1000 steps)	High	Excellent	Slow (~1000 NFE)	Good	Yes (standard)
DDIM	Same as DDPM	High	Good (less diverse)	Fast (20-50 NFE)	Good	Yes (sampler only)
Flow Matching	Fast	Very High	Excellent	Very Fast (4-8 NFE)	Very Good	Yes - clean integration
Rectified Flow	Very Fast	High	Very Good	Fastest (1-4 NFE)	Excellent	Yes - recommended
CFM	Fast	Very High	Excellent	Fast (4-8 NFE)	Excellent	Yes - best for conditioning

2.3 Should You Switch from DDPM to Flow Matching?

> *Recommendation: YES — migrate to Conditional Flow Matching (CFM) with a Rectified Flow transport path.*

For 1D ECG generation specifically, CFM offers three decisive advantages over DDPM+DDIM:

1. Diversity by construction: CFM samples are inherently stochastic at inference time. Unlike DDIM, CFM does not suppress variability through deterministic denoising. This directly addresses your HR std collapse.
2. Straight ODE paths: For 1D signals, straight interpolation between noise and ECG does not introduce morphological artifacts. Curved paths (DDPM) can generate non-physiological transients.
3. Faster inference: CFM requires 4-8 function evaluations (NFE) vs DDPM's 1000 (or 50 with DDIM). This enables real-time generation of patient-specific ECGs.

DiT Compatibility

CFM replaces only the training objective — the noise prediction loss $L = ||e_{theta}(x_t, t) - e||^2$ becomes a velocity prediction loss $L = ||v_{theta}(x_t, t) - (x_1 - x_0)||^2$. Your DiT backbone remains identical. The key changes are:

- Replace DDPM forward process with linear interpolation: $x_t = (1-t)*x_0 + t*x_1$
- Change training target from epsilon to velocity: $v_{target} = x_1 - x_0$
- Replace DDIM sampler with an ODE solver (Euler or RK4 with 4-8 steps)
- Add time embedding for t in [0,1] continuous (already present in DiT)

Compute Differences

Training cost is similar (~10-15% faster for CFM due to simpler loss landscape). Inference cost drops by 95% (1000 steps -> 8 steps). For 2500-sample ECG sequences, CFM generation becomes effectively real-time.

3. Recent Research Papers (2024-2025)

3.1 ECGTwin (2025)

Field	Details
Year	2025
Core Idea	Patient-specific ECG digital twin generation using reference signal conditioning. Addresses personalized generation without ground truth.
Architecture	Latent diffusion model with cross-attention conditioning on clinical text tokens, heart rate, age, sex. Reference ECG encoded via nomic-embed-text-v1.5 text embeddings.
Losses	Diffusion MSE in latent space; semantic alignment via cross-attention on clinical report tokens.
Key Results	Outperforms DiffuSETS-p baseline on HR-MAE and FID. First personalized ECG digital twin system.
Link	arxiv.org/abs/2508.02720

3.2 DiffuSETS (2025)

Field	Details
Year	2025 (Patterns, Cell Press)

Core Idea	12-lead ECG generation conditioned on clinical text reports and patient metadata (age, sex, HR). First diffusion model trained on MIMIC-IV-ECG for text-to-ECG generation.
Architecture	VAE for signal compression; LLM semantic encoder for text reports; DDPM denoising in latent space with cross-modal cross-attention.
Losses	DDPM epsilon loss in latent space. Clinical text alignment via cross-attention. Three-level evaluation: signal, feature, diagnostic levels.
Key Results	Over 90% accuracy vs real ECG patterns. HR conditioning directly injectable as scalar input. Supports Turing-test-level fidelity.
Link	arxiv.org/abs/2501.05932 | cell.com/patterns/fulltext/S2666-3899(25)00139-4

3.3 MIDT-ECG (2025)

Field	Details
Year	2025
Core Idea	Mel-Spectrogram Informed Diffusion Training adds time-frequency domain supervision to enforce physiological morphological realism. Directly addresses the MSE-only loss limitation.
Architecture	SSSD-ECG backbone (S4 state-space model) with additional demographic conditioning (age, sex). Mel-spectrogram computed on ECG and used as auxiliary training signal.
Losses	DDPM epsilon loss + Mel-spectrogram MSE loss. The spectrogram loss enforces P-QRS-T morphological structure without needing explicit beat detection.
Key Results	Reduces interlead correlation error by 74%. Improves morphological coherence. Privacy metrics exceed baseline by 4-8%.
Link	arxiv.org/abs/2510.05492

3.4 CardioFlow / PPGFlowECG (2025)

Field	Details
Year	2025
Core Idea	PPG-to-ECG translation using Rectified Flow. Learns a velocity field from noise to ECG signal conditioned on PPG. Enables 1-few step generation. Probabilistic QRS peak masking emphasizes morphologically critical regions.
Architecture	Latent Rectified Flow with cross-modal PPG encoder. QRS peak masking as attention weighting during training.
Losses	Velocity MSE (rectified flow) + QRS-weighted morphology loss via probabilistic peak masking.
Key Results	Achieves 1-step generation. Improves QRS morphology vs diffusion baselines. Demonstrates feasibility of flow matching for ECG.
Link	arxiv.org/abs/2509.19774

3.5 UniCardio (2025)

Field	Details

Year	2025 (Nature Machine Intelligence)
Core Idea	Multimodal diffusion transformer for joint ECG/PPG/blood pressure generation and reconstruction. Continual learning for modality combination.
Architecture	Diffusion Transformer (DiT) backbone with multi-modal token mixing. This is the closest architecture to your setup.
Losses	DDPM epsilon loss with modality-specific decoders. Cross-modal alignment loss for cardiovascular signal consistency.
Key Results	Outperforms task-specific baselines in signal denoising, imputation, and translation. Published in Nature MI.
Link	nature.com/articles/s42256-025-01147-y

3.6 DiffECG (2023/2024)

Field	Details
Year	2023 (updated 2024)
Core Idea	Versatile DDPM for ECG synthesis covering heartbeat generation, partial imputation, and full forecasting. First generalized conditional approach for ECG synthesis.
Architecture	1D U-Net based DDPM with class conditioning. Handles single-lead and multi-lead.
Losses	Standard DDPM MSE. No explicit morphology losses reported.
Key Results	Outperforms GAN-based ECG generators on FID and downstream classifier performance.
Link	arxiv.org/abs/2306.01875

4. ECG-Specific Loss Functions

The current loss suite (MSE + FFT + Pearson + cosine) optimizes global signal fidelity but lacks physiological constraint enforcement. The following losses target ECG morphology, rhythm regularity, and interval preservation.

4.1 Loss Function Rationale

RR Interval Consistency Loss

RR intervals encode HR and its variability (HRV). A model that collapses HR diversity is producing signals where RR intervals cluster tightly. This loss penalizes inter-beat inconsistency and distribution mismatch with the training data.

QT Interval Preservation Loss

The QT interval (Q-wave onset to T-wave end) represents ventricular repolarization time. Prolonged QT is a critical arrhythmia risk marker. The model should preserve QT proportional to RR (via Bazett's formula: QTc = QT / sqrt(RR)).

P-QRS-T Morphology Loss

ECG waveforms consist of P waves (atrial depolarization), QRS complexes (ventricular depolarization), and T waves (ventricular repolarization). A morphology loss ensures each wave component has the correct amplitude, duration, and relative timing.

Dynamic Time Warping (DTW) Loss

DTW provides an elastic distance measure between waveforms that is invariant to local temporal shifts. Unlike MSE, DTW correctly penalizes beat phase misalignments without over-penalizing minor timing jitter. Differentiable DTW (via soft-DTW) enables backpropagation.

Spectral Coherence Loss

While FFT MSE penalizes spectral magnitude differences, spectral coherence (cross-spectral density normalized by auto-spectra) measures the phase relationship between generated and real signals. This is particularly important for preserving the spectral fingerprint of patient-specific HRV patterns.

4.2 PyTorch Implementations

- *All implementations assume ECG tensor shape: (batch_size, 2500) at 500 Hz sampling rate.*

Loss 1: RR Interval Consistency Loss

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from scipy.signal import find_peaks
import numpy as np

class RRIntervalLoss(nn.Module):
    """
    RR interval loss using differentiable R-peak detection via
    local maximum detection with soft-peak approximation.
    Penalizes: (1) RR interval variance mismatch, (2) mean HR mismatch.
    """
    def __init__(self, fs=500, hr_weight=1.0, var_weight=2.0):
        super().__init__()
        self.fs = fs
        self.hr_weight = hr_weight
        self.var_weight = var_weight
        # Gaussian smoothing for soft peak detection
        sigma = 5  # samples
        kernel_size = 4 * sigma + 1
        x = torch.arange(kernel_size).float() - kernel_size // 2
        gauss = torch.exp(-x**2 / (2 * sigma**2))
        self.register_buffer('gauss_kernel',
            gauss.view(1, 1, -1) / gauss.sum())

    def soft_rpeaks(self, ecg):
        """Differentiable R-peak probability map via template correlation."""
        B, T = ecg.shape
        x = ecg.unsqueeze(1)  # (B, 1, T)
        # Smooth signal
        smoothed = F.conv1d(x, self.gauss_kernel, padding=self.gauss_kernel.shape[-1]//2)
```

```
        smoothed = smoothed.squeeze(1)  # (B, T)
        # Local maxima via soft-max over sliding window
        win = 100  # ~200ms at 500Hz (min RR for 300bpm)
        peaks = F.max_pool1d(smoothed.unsqueeze(1), win, stride=1,
                             padding=win//2).squeeze(1)
        # Soft peak map: high where signal == local max
        peak_map = torch.sigmoid(10.0 * (smoothed - peaks + 1e-4))
        return peak_map  # (B, T), values in [0,1]

    def estimate_hr_from_peaks(self, peak_map):
        """Estimate HR in bpm from soft peak map."""
        B, T = peak_map.shape
        # Weighted center-of-mass for inter-peak spacing
        t = torch.arange(T, device=peak_map.device).float()
        # Mean peak interval via autocorrelation of peak_map
        fft_peaks = torch.fft.rfft(peak_map, dim=-1)
        autocorr = torch.fft.irfft(fft_peaks * fft_peaks.conj(), dim=-1, n=T)
        # Find first secondary peak in autocorrelation (lag > 100 samples)
        autocorr_crop = autocorr[:, 100:]
        best_lag = torch.argmax(autocorr_crop, dim=-1).float() + 100
        hr = (self.fs / best_lag) * 60.0  # bpm
        return hr

    def forward(self, generated, real):
        # Normalize to [-1, 1] range for stable peak detection
        gen_norm = (generated - generated.mean(-1, keepdim=True)) / (
            generated.std(-1, keepdim=True) + 1e-6)
        real_norm = (real - real.mean(-1, keepdim=True)) / (
            real.std(-1, keepdim=True) + 1e-6)

        gen_peaks = self.soft_rpeaks(gen_norm)
        real_peaks = self.soft_rpeaks(real_norm)

        gen_hr = self.estimate_hr_from_peaks(gen_peaks)
        real_hr = self.estimate_hr_from_peaks(real_peaks)

        # Loss 1: mean HR alignment
        hr_loss = F.l1_loss(gen_hr, real_hr)

        # Loss 2: HR variance matching across batch
        gen_hr_std = gen_hr.std()
        real_hr_std = real_hr.std()
        var_loss = F.mse_loss(gen_hr_std, real_hr_std)

        return self.hr_weight * hr_loss + self.var_weight * var_loss
```

Loss 2: QT Interval Loss

```
class QTIntervalLoss(nn.Module):
    """
    Enforces QTc (corrected QT interval) via Bazett formula: QTc = QT/sqrt(RR).
    Uses spectral proxy for QT duration (energy in 0-15Hz band / total energy).
    A longer low-freq dominated burst after QRS = longer QT.
    """
```

```
    def __init__(self, fs=500):
        super().__init__()
        self.fs = fs
        self.low_freq_cutoff = 15  # Hz

    def spectral_qt_proxy(self, ecg):
        """Proxy for QT duration: ratio of low-freq to total power."""
        B, T = ecg.shape
        fft = torch.fft.rfft(ecg, dim=-1)
        freqs = torch.fft.rfftfreq(T, d=1.0/self.fs).to(ecg.device)
        mag = fft.abs() ** 2
        low_mask = freqs < self.low_freq_cutoff
        low_power = mag[:, low_mask].sum(-1)
        total_power = mag.sum(-1) + 1e-8
        return low_power / total_power

    def forward(self, generated, real):
        gen_qt = self.spectral_qt_proxy(generated)
        real_qt = self.spectral_qt_proxy(real)
        # Match distribution of QT proxies
        loss = F.mse_loss(gen_qt, real_qt)
        # Penalize extreme QT values (QTc > 0.44s flags arrhythmia risk)
        max_qt = 0.44 * self.fs / 500  # normalized
        prolongation_penalty = F.relu(gen_qt - 0.70).mean()  # empirical threshold
        return loss + 0.5 * prolongation_penalty
```

Loss 3: P-QRS-T Morphology Loss

```
class MorphologyLoss(nn.Module):
    """
    P-QRS-T morphology enforcement via frequency band decomposition.
    - P-wave: 0.5-5 Hz band energy and timing
    - QRS complex: 5-40 Hz band energy (fast transient)
    - T-wave: 0.5-8 Hz band energy
    Additionally enforces beat template consistency across the signal.
    """
    def __init__(self, fs=500):
        super().__init__()
        self.fs = fs
        # Frequency bands for each ECG component
        self.bands = {
            'p_wave':  (0.5, 5.0),
            'qrs':     (5.0, 40.0),
            't_wave':  (0.5, 8.0),
            'baseline': (0.05, 0.5),
        }

    def bandpass_energy(self, ecg, low, high):
        """Compute energy in [low, high] Hz band."""
        B, T = ecg.shape
        freqs = torch.fft.rfftfreq(T, d=1.0/self.fs).to(ecg.device)
        fft = torch.fft.rfft(ecg, dim=-1)
        mask = ((freqs >= low) & (freqs <= high)).float()
        band_energy = (fft.abs() ** 2 * mask).sum(-1) / (T + 1e-8)
        return band_energy
```

```
    def beat_template_consistency(self, ecg, n_segments=5):
        """Check that beat morphology is consistent across signal."""
        B, T = ecg.shape
        seg_len = T // n_segments
        segments = ecg[:, :seg_len * n_segments].reshape(B, n_segments, seg_len)
        # Pairwise cosine similarity between segments
        seg_norm = F.normalize(segments, dim=-1)
        # Mean similarity across adjacent pairs
        sim = (seg_norm[:, :-1] * seg_norm[:, 1:]).sum(-1)  # (B, n_segments-1)
        consistency = sim.mean()
        return -consistency  # want high similarity -> negative as loss

    def forward(self, generated, real):
        total_loss = 0.0
        for name, (low, high) in self.bands.items():
            gen_energy = self.bandpass_energy(generated, low, high)
            real_energy = self.bandpass_energy(real, low, high)
            weight = 2.0 if name == 'qrs' else 1.0  # QRS most critical
            total_loss += weight * F.mse_loss(gen_energy, real_energy)

        # Beat template consistency
        total_loss += 0.5 * self.beat_template_consistency(generated)
        return total_loss
```

Loss 4: Soft-DTW Waveform Alignment Loss

```
class SoftDTWLoss(nn.Module):
    """
    Differentiable DTW loss for waveform alignment.
    Uses soft-DTW with gamma parameter controlling smoothness.
    Applied on downsampled ECG beats for computational efficiency.
    """
    def __init__(self, gamma=0.1, normalize=True, downsample=4):
        super().__init__()
        self.gamma = gamma
        self.normalize = normalize
        self.downsample = downsample

    def pairwise_distances(self, x, y):
        """Pairwise L2 distances between x (B,T,1) and y (B,T,1)."""
        x2 = x.pow(2).sum(-1, keepdim=True)  # (B, T, 1)
        y2 = y.pow(2).sum(-1, keepdim=True).transpose(1, 2)  # (B, 1, T)
        xy = torch.bmm(x, y.transpose(1, 2))  # (B, T, T)
        return torch.clamp(x2 + y2 - 2*xy, min=0).sqrt()

    def soft_dtw(self, D):
        """Compute soft-DTW cost matrix via dynamic programming."""
        B, N, M = D.shape
        device = D.device
        # Initialize R with large values
        R = torch.full((B, N+2, M+2), float('inf'), device=device)
        R[:, 0, 0] = 0.0
        for i in range(1, N+1):
            for j in range(1, M+1):
```

```
                r0 = R[:, i-1, j-1]
                r1 = R[:, i-1, j]
                r2 = R[:, i, j-1]
                # Soft minimum via log-sum-exp
                rmin = -self.gamma * torch.logsumexp(
                    torch.stack([-r0/self.gamma, -r1/self.gamma, -r2/self.gamma], dim=-
1),
                    dim=-1
                )
                R[:, i, j] = D[:, i-1, j-1] + rmin
        return R[:, N, M]

    def forward(self, generated, real):
        B, T = generated.shape
        # Downsample for efficiency
        ds = self.downsample
        gen_ds = generated[:, ::ds].unsqueeze(-1)
        real_ds = real[:, ::ds].unsqueeze(-1)
        # Pairwise distances
        D = self.pairwise_distances(gen_ds, real_ds)
        # Soft-DTW
        dtw_cost = self.soft_dtw(D)
        if self.normalize:
            dtw_cost = dtw_cost / (T // ds)
        return dtw_cost.mean()
```

Loss 5: Spectral Coherence Loss

```
class SpectralCoherenceLoss(nn.Module):
    """
    Spectral coherence loss measures the phase consistency between
    generated and real ECG signals across the HRV-relevant frequency
    bands (VLF: 0.003-0.04Hz, LF: 0.04-0.15Hz, HF: 0.15-0.4Hz).
    Also penalizes magnitude spectrum mismatch via log-scale MSE.
    """
    def __init__(self, fs=500, nfft=None):
        super().__init__()
        self.fs = fs
        self.nfft = nfft
        # HRV frequency bands (Hz)
        self.hrv_bands = [(0.003, 0.04), (0.04, 0.15), (0.15, 0.4)]

    def forward(self, generated, real):
        B, T = generated.shape
        nfft = self.nfft or T

        gen_fft = torch.fft.rfft(generated, n=nfft, dim=-1)
        real_fft = torch.fft.rfft(real, n=nfft, dim=-1)
        freqs = torch.fft.rfftfreq(nfft, d=1.0/self.fs).to(generated.device)

        # Cross-spectral density
        Sxy = gen_fft * real_fft.conj()  # (B, F)
        Sxx = gen_fft * gen_fft.conj()   # (B, F)
        Syy = real_fft * real_fft.conj() # (B, F)
```

```
        # Coherence: |Sxy|^2 / (Sxx * Syy) in [0,1]
        coherence = Sxy.abs()**2 / (Sxx.abs() * Syy.abs() + 1e-10)

        # Loss: 1 - mean coherence (want coherence close to 1)
        coherence_loss = (1.0 - coherence.real).mean()

        # Log-magnitude spectrum MSE
        gen_logmag = torch.log1p(gen_fft.abs())
        real_logmag = torch.log1p(real_fft.abs())
        mag_loss = F.mse_loss(gen_logmag, real_logmag)

        return coherence_loss + 0.5 * mag_loss
```

Combined ECG Loss

```
class ECGPhysiologicalLoss(nn.Module):
    """Combined physiological loss suite for ECG DiT training."""
    def __init__(self, fs=500):
        super().__init__()
        self.rr_loss = RRIntervalLoss(fs=fs)
        self.qt_loss = QTIntervalLoss(fs=fs)
        self.morph_loss = MorphologyLoss(fs=fs)
        self.dtw_loss = SoftDTWLoss()
        self.coherence_loss = SpectralCoherenceLoss(fs=fs)

    def forward(self, generated, real, weights=None):
        if weights is None:
            weights = {'rr': 1.0, 'qt': 0.5, 'morph': 1.5,
                       'dtw': 0.3, 'coherence': 1.0}

        losses = {
            'rr':        self.rr_loss(generated, real),
            'qt':        self.qt_loss(generated, real),
            'morph':     self.morph_loss(generated, real),
            'dtw':       self.dtw_loss(generated, real),
            'coherence': self.coherence_loss(generated, real),
        }
        total = sum(weights[k] * v for k, v in losses.items())
        return total, losses
```

5. Increasing Heart Rate Diversity

5.1 Root Cause Analysis

The HR std collapse from 47 bpm to 21 bpm is a symptom of mode collapse in the conditional distribution. Multiple interacting causes:

Cause	Mechanism
DDIM determinism	DDIM's deterministic reverse process maps each noise seed to a fixed output. With a fixed noise schedule and strong conditioning, all seeds map to near-identical HR modes.

Regression to mean via MSE	MSE loss minimizes expected value, which for multi-modal distributions (varied HR) converges to the mean, suppressing tails.
CFG guidance scale too high	High classifier-free guidance weight sharply concentrates the output distribution around the conditional mode.
HR not explicitly conditioned	Without HR as an explicit conditioning signal, the model must infer HR from patient embedding alone. Patient embeddings may not encode HR variability sufficiently.
Training data imbalance	If the training set has few examples of very high or very low HR, the model cannot generate them.

5.2 Conditioning Strategies

Explicit HR Conditioning

The single most impactful fix is to inject HR as an explicit conditioning signal. Normalize HR to [0,1] range across the dataset and project it into the DiT token embedding space.

```
class HRConditioner(nn.Module):
    """Injects heart rate as a continuous conditioning embedding."""
    def __init__(self, d_model, hr_min=30, hr_max=200):
        super().__init__()
        self.hr_min = hr_min
        self.hr_max = hr_max
        self.hr_embed = nn.Sequential(
            nn.Linear(1, d_model // 4),
            nn.SiLU(),
            nn.Linear(d_model // 4, d_model)
        )
        # HR bin embedding for discrete bucketing (optional, improves coverage)
        self.n_bins = 30  # 30 HR bins from 30-200 bpm
        self.hr_bin_embed = nn.Embedding(self.n_bins, d_model)

    def forward(self, hr_bpm):
        # Normalize HR to [0,1]
        hr_norm = (hr_bpm - self.hr_min) / (self.hr_max - self.hr_min)
        hr_norm = hr_norm.clamp(0, 1).unsqueeze(-1)  # (B, 1)
        # Continuous embedding
        cont_embed = self.hr_embed(hr_norm)  # (B, d_model)
        # Discrete bin embedding (for HR coverage)
        hr_bin = (hr_norm * (self.n_bins - 1)).long().squeeze(-1)
        bin_embed = self.hr_bin_embed(hr_bin)  # (B, d_model)
        return cont_embed + bin_embed

# In DiT forward pass, add HR conditioning to time embedding:
# t_emb = time_embedding(t) + hr_conditioner(hr_bpm)
# This makes HR a first-class citizen alongside the timestep.
```

RR Interval Conditioning

For fine-grained rhythm control, condition on the target RR interval sequence directly. This enables generation of ECGs with specific HRV patterns.

```
class RRSequenceConditioner(nn.Module):
    """Encodes target RR interval sequence into conditioning embeddings."""
```

```
    def __init__(self, d_model, max_beats=30):
        super().__init__()
        self.max_beats = max_beats
        # Transformer encoder for RR sequence
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=4, dim_feedforward=d_model*2,
            dropout=0.1, batch_first=True
        )
        self.rr_encoder = nn.TransformerEncoder(encoder_layer, num_layers=2)
        self.rr_proj = nn.Linear(1, d_model)

    def forward(self, rr_intervals):
        # rr_intervals: (B, max_beats) in seconds
        x = self.rr_proj(rr_intervals.unsqueeze(-1))  # (B, T, d_model)
        encoded = self.rr_encoder(x)  # (B, T, d_model)
        # Pool to single conditioning vector
        cond = encoded.mean(dim=1)  # (B, d_model)
        return cond
```

5.3 Data Augmentation

Stochastic HR Perturbation

```
def hr_perturbation_augment(ecg, hr_bpm, sigma=15.0, fs=500):
    """
    Time-warp ECG signal to simulate different heart rates.
    Resamples the signal to match a perturbed HR target.
    ecg: (B, 2500)
    hr_bpm: (B,) current HR for each sample
    """
    B, T = ecg.shape
    target_hr = hr_bpm + sigma * torch.randn_like(hr_bpm)
    target_hr = target_hr.clamp(40, 200)  # physiological range
    scale = hr_bpm / target_hr  # time-scaling factor

    augmented = []
    for i in range(B):
        s = scale[i].item()
        new_len = int(T * s)
        if new_len < 100 or new_len > T * 3:
            augmented.append(ecg[i])
            continue
        # Resample via interpolation
        resampled = F.interpolate(
            ecg[i:i+1].unsqueeze(0),  # (1, 1, T)
            size=new_len,
            mode='linear', align_corners=True
        ).squeeze()  # (new_len,)
        # Crop or pad to original length
        if new_len >= T:
            aug = resampled[:T]
        else:
            pad = T - new_len
            aug = F.pad(resampled, (0, pad), mode='replicate')
        augmented.append(aug)
```

```
    return torch.stack(augmented), target_hr
```

5.4 Model Modifications

Stochastic Sampling (DDPM instead of DDIM)

The simplest fix for DDIM collapse: switch back to stochastic DDPM sampling or use DDIM with eta > 0 to re-inject stochasticity.

```
# In DDIM sampling, set eta > 0 to add stochasticity:
# sigma_t = eta * sqrt((1-alpha_prev)/(1-alpha)) * sqrt(1-alpha/alpha_prev)
# x_prev = sqrt(alpha_prev) * pred_x0
#        + sqrt(1-alpha_prev-sigma_t**2) * pred_noise
#        + sigma_t * torch.randn_like(x)   # <-- stochastic term

# Use eta = 0.5 to 1.0 as a starting point.
# eta=0: pure DDIM (deterministic, low diversity)
# eta=1: equivalent to DDPM (fully stochastic, max diversity)
```

HR Variance Regularization Loss

```
class HRVarianceRegularizationLoss(nn.Module):
    """
    Penalizes the model when the batch-level HR standard deviation
    is lower than a target value derived from the training data.
    This is applied to the GENERATOR during training.
    """
    def __init__(self, target_hr_std=47.0, weight=2.0):
        super().__init__()
        self.target_hr_std = target_hr_std
        self.weight = weight

    def forward(self, generated_hr_estimates):
        # generated_hr_estimates: (B,) HR estimated from generated signals
        batch_std = generated_hr_estimates.std()
        # Penalize if std is below target
        diversity_loss = F.relu(self.target_hr_std - batch_std)
        return self.weight * diversity_loss

class RRDistributionMatchingLoss(nn.Module):
    """
    Matches the distribution of RR intervals via maximum mean discrepancy (MMD)
    between generated and real RR interval distributions.
    """
    def __init__(self, kernel_bw=0.5):
        super().__init__()
        self.kernel_bw = kernel_bw

    def rbf_kernel(self, x, y):
        diff = x.unsqueeze(1) - y.unsqueeze(0)  # (N, M)
        return torch.exp(-diff**2 / (2 * self.kernel_bw**2))

    def mmd(self, x, y):
```

```
        """Maximum Mean Discrepancy between x and y distributions."""
        Kxx = self.rbf_kernel(x, x).mean()
        Kyy = self.rbf_kernel(y, y).mean()
        Kxy = self.rbf_kernel(x, y).mean()
        return Kxx + Kyy - 2 * Kxy

    def forward(self, gen_rr, real_rr):
        # gen_rr, real_rr: (B,) mean RR interval in seconds
        return self.mmd(gen_rr, real_rr)
```

5.5 Classifier-Free Guidance Tuning

If using CFG with guidance scale w, the effective generation distribution is: p_guided(x|c) proportional to p(x|c)^(1+w) * p(x)^(-w). High w concentrates probability mass. For maximum HR diversity:

- Reduce guidance scale from typical 7.5 to 2.0-4.0 at inference time
- Use dynamic guidance: lower w for diverse generation, higher w for specific HR targets
- Apply guidance only to patient identity (not HR), letting HR vary freely

```
def guided_sample(model, noise, patient_cond, hr_cond, w_patient=3.0, w_hr=5.0):
    """
    Separate guidance scales for patient identity vs HR conditioning.
    High w_hr enforces the target HR; low w_patient allows morphological diversity.
    """
    # Conditional predictions
    eps_cond = model(noise, t, patient_cond, hr_cond)
    eps_uncond = model(noise, t, None, None)
    eps_hr_only = model(noise, t, None, hr_cond)

    # Decomposed guidance
    eps_guided = eps_uncond \
        + w_patient * (eps_hr_only - eps_uncond) \
        + w_hr * (eps_cond - eps_hr_only)
    return eps_guided
```

6. Practical Recommendations & Roadmap

6.1 Priority Action Plan

Priority	Action	Expected Impact	Effort
1	Add explicit HR conditioning	Directly controls HR at generation time. Enables diversity by sampling HR from the real distribution before generation.	Low (1-2 days)
2	Switch DDIM eta=0 to eta=0.7	Restores stochasticity to sampling. Free fix - no retraining required. Expected to increase HR std by 50-100%.	Trivial (< 1 hour)
3	Add RR variance loss	Directly penalizes HR collapse at training time. Use weight=2.0 initially and tune.	Low (0.5 days)
4	Add morphology loss (Loss 3)	Enforces P-QRS-T structure. Reduces morphological artifacts without needing beat detection.	Low (1 day)

5	Add HR augmentation	Time-warping augmentation exposes model to full HR range even if training data is imbalanced.	Low (0.5 days)
6	Migrate to CFM (medium-term)	Fundamentally fixes diversity, speeds up inference 50x, enables HR-conditional sampling with guaranteed diversity.	Medium (1-2 weeks)
7	Add DTW + coherence loss	Fine-tunes temporal alignment. Use only after basic morphology and HR issues are resolved.	Medium (2-3 days)

6.2 Which Model to Use

Short term (current sprint): Keep DDPM backbone, fix DDIM eta, add HR conditioning and RR variance loss. This alone should recover most of the HR diversity.

Medium term (next sprint): Migrate to Conditional Flow Matching. Replace DDPM loss with CFM velocity loss. No architecture change required. Benefits: faster inference, better diversity, cleaner conditioning.

Long term: Consider Rectified Flow with 1-2 step distillation for real-time deployment. Use latent DiT (VAE compression) to reduce sequence length from 2500 to 312 (8x) for faster training.

6.3 Suggested Training Pipeline

```
# Recommended training loop for ECG DiT with CFM + physiological losses

class ECGDiTTrainer:
    def __init__(self, model, fs=500):
        self.model = model
        self.physio_loss = ECGPhysiologicalLoss(fs=fs)
        self.hr_var_loss = HRVarianceRegularizationLoss(target_hr_std=47.0)
        self.hr_conditioner = HRConditioner(d_model=model.d_model)

    def cfm_loss(self, x0, cond):
        """Conditional Flow Matching training objective."""
        B = x0.shape[0]
        # Sample time uniformly
        t = torch.rand(B, device=x0.device)
        # Sample noise
        x1 = torch.randn_like(x0)
        # Interpolate: x_t = (1-t)*x1 + t*x0  (noise->data direction)
        t_expanded = t.view(B, 1)
        xt = (1 - t_expanded) * x1 + t_expanded * x0
        # Target velocity (noise->data = x0-x1)
        v_target = x0 - x1
        # Predict velocity
        v_pred = self.model(xt, t, cond)
        return F.mse_loss(v_pred, v_target)

    def training_step(self, batch):
        ecg, patient_id, hr_bpm = batch

        # HR augmentation (50% of the time)
        if torch.rand(1) > 0.5:
            ecg, hr_bpm = hr_perturbation_augment(ecg, hr_bpm, sigma=15.0)
```

```
# Build conditioning
hr_emb = self.hr_conditioner(hr_bpm)
patient_emb = self.model.patient_encoder(patient_id)
cond = hr_emb + patient_emb

# CFM loss (or DDPM epsilon loss if keeping DDPM)
loss_cfm = self.cfm_loss(ecg, cond)

# Generate samples for physiological loss
with torch.no_grad():
    gen_ecg = self.generate(cond, n_steps=8)

# Physiological losses on generated samples
loss_physio, physio_dict = self.physio_loss(gen_ecg, ecg)

# HR variance regularization
gen_hr = self.estimate_hr(gen_ecg)
loss_hr_var = self.hr_var_loss(gen_hr)

# Total loss
total = loss_cfm + 0.3 * loss_physio + loss_hr_var
return total, {
    'cfm': loss_cfm, 'physio': loss_physio,
    'hr_var': loss_hr_var, **physio_dict
}
```

6.4 Evaluation Metrics

Use the three-level evaluation framework from DiffuSETS (Lai et al., 2025):

- Signal level: FID (Frechet Inception Distance on ECG features), Precision/Recall, DTW distance
- Feature level: HR-MAE (heart rate mean absolute error), HR-Std ratio (target: gen_std/real_std > 0.8), QTc distribution overlap
- Diagnostic level: Train classifier on synthetic ECGs, test on real; target >85% of real-data performance
- HR diversity: Monitor both mean HR error (<5 bpm) and std HR ratio (>0.85) separately

7. References

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. NeurIPS 33, 6840-6851.

Song, J., Meng, C., & Ermon, S. (2020). Denoising Diffusion Implicit Models. ICLR 2021.

Lipman, Y., Chen, R., Ben-Hamu, H., Nickel, M., & Le, M. (2022). Flow matching for generative modeling. ICLR 2023.

Liu, X., Gong, C., & Liu, Q. (2022). Flow straight and fast: Learning to generate and transfer data with rectified flow. ICLR 2023.

Albergo, M.S. & Vanden-Eijnden, E. (2022). Building normalizing flows with stochastic interpolants. ICLR 2023.

Lai, Y. et al. (2025). DiffuSETS: 12-lead ECG generation conditioned on clinical text reports and patient-specific information. Patterns. arXiv:2501.05932

[ECGTwin] (2025). Personalized ECG generation using controllable diffusion model. arXiv:2508.02720

[MIDT-ECG] (2025). High-fidelity synthetic ECG generation via mel-spectrogram informed diffusion training. arXiv:2510.05492

[PPGFlowECG] (2025). Latent rectified flow with cross-modal encoding for PPG-guided ECG generation. arXiv:2509.19774

[UniCardio] (2025). Versatile cardiovascular signal generation with a unified diffusion transformer. Nature Machine Intelligence.

[DiffECG] Ben-Hamadou, A. et al. (2023/2024). DiffECG: A versatile probabilistic diffusion model for ECG signal synthesis. arXiv:2306.01875

[SSSD-ECG] Alcaraz, J.M.L. & Strodthoff, N. (2023). Diffusion-based conditional ECG generation with structured state space models. Computers in Biology and Medicine, 163, 107115.

Nichol, A. & Dhariwal, P. (2021). Improved denoising diffusion probabilistic models. ICML 2021.